



INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS
SOCIALES Y ADMINISTRATIVAS

Abstracción y uso de Datos

Reporte Final

Secuencia:3CM30

Integrantes del equipo:

Grande Quintana Jorge Daniel

Herrera Quiroz Aolani Tziranda

Verboonen Sierra Edgar Jaret

Moreno Sánchez Evelyn Dennise

GRAFO

1. Introducción

Este reporte describe el funcionamiento de un programa escrito en lenguaje C++ que representa un conjunto de ciudades mexicanas y las conexiones que existen entre ellas, utilizando una estructura de datos conocida como grafo. En este tipo de estructura, una conexión entre dos ciudades funciona en ambos sentidos. Es decir, si una ciudad se conecta con otra, se considera que el camino puede recorrerse desde cualquiera de las dos ciudades hacia la otra. El programa permite al usuario consultar la información de las ciudades y sus conexiones, agregar o eliminar elementos, buscar una ruta entre dos ciudades, buscar una ciudad por su nombre, y guardar toda la información en distintos tipos de archivo. Internamente, fue construido aplicando Programación Orientada a Objetos, lo cual se detalla más adelante en este documento.

2. ¿Qué es un Grafo?

Un grafo es una forma de representar un conjunto de elementos junto con las relaciones que existen entre ellos. En este programa, los elementos son ciudades, y las relaciones son las conexiones que existen entre dichas ciudades. La característica principal de un grafo es que sus conexiones no tienen una dirección definida. Si Cancún se conecta con la Ciudad de México, esa misma conexión permite ir de Cancún hacia la Ciudad de México y también de la Ciudad de México hacia Cancún.

Ejemplo

Cancun <---> Ciudad de Mexico

Esta forma de representar información resulta útil para modelar situaciones donde una relación es mutua, como una carretera que puede recorrerse en cualquiera de los dos sentidos, o una amistad entre dos personas.

3. Ciudades y conexiones Iniciales

Al iniciar su ejecución, el programa carga automáticamente cinco ciudades mexicanas junto con un conjunto de conexiones entre ellas, de manera que el usuario puede comenzar a explorar el programa de inmediato, sin necesidad de capturar información manualmente.

Número	Ciudad	Estado
1	Cancún	Quintana Roo
2	Ciudad de México	CDMX
3	Guadalajara	Jalisco
4	Baja California	Baja California
5	Chiapas	Chiapas

Las conexiones precargadas:

- Cancún se conecta con Ciudad de México.
- Ciudad de México se conecta con Guadalajara.
- Guadalajara se conecta con Baja California.
- Baja California se conecta con Ciudad de México.
- Ciudad de México se conecta con Chiapas.
- Cancún se conecta con Chiapas.

4. Funciones del Menú

Ver ciudades

Muestra el listado completo de las ciudades registradas, junto con su número y su estado correspondiente.

Ver conexiones

Muestra todas las conexiones existentes en el programa, indicando entre qué ciudades se establece cada una.

Buscar ruta

Permite al usuario indicar dos ciudades, una de origen y una de destino, para que el programa determine si existe una conexión entre ellas. Primero se revisa si la conexión es directa, si no lo es, el programa intenta encontrar una ruta que pase por una ciudad intermedia.

Ejemplo de búsqueda directa

```
Ruta: Cancun --> Chiapas
Directo: Cancun <---> Chiapas
```

Ejemplo de búsqueda con escala

```
Ruta: Guadalajara --> Chiapas
Con escala en Ciudad de Mexico:
Guadalajara <---> Ciudad de Mexico
Ciudad de Mexico <---> Chiapas
```

Agregar ciudad y/o conexión

Permite registrar una nueva ciudad, una nueva conexión, o ambas en la misma operación. Al agregar una ciudad, el programa le asigna automáticamente un número de identificación. Al agregar una conexión, se solicitan los números de las dos ciudades que se desean conectar.

Eliminar ciudad y/o conexión

Permite eliminar una ciudad junto con todas las conexiones que tenga registradas, o bien eliminar únicamente una conexión en particular sin afectar a las ciudades involucradas.

Buscar ciudad

Solicita al usuario el nombre de una ciudad. Si la ciudad existe, el programa la muestra junto con todas sus conexiones. Si la ciudad no existe, el programa lo informa y pregunta al usuario si desea registrarla en ese momento, ofreciendo además la posibilidad de agregarle una conexión.

Ejemplo cuando la ciudad no existe

```
La ciudad "Monterrey" no existe.
¿Desea crearla? (s/n):
```

Exportar información

Las últimas opciones del menú permiten guardar la información actual del programa en distintos tipos de archivo, de manera individual o todos a la vez. Esta función se describe con más detalle en la sección 6 de este documento.

5. Archivos Generados

El programa permite guardar la información de las ciudades y sus conexiones en cuatro tipos de archivo distintos. Cada vez que el usuario agrega o elimina algún elemento, los archivos se actualizan de manera automática.

Archivo de texto

Es un archivo simple, fácil de leer directamente, donde se enlistan las ciudades y sus conexiones.

CIUDADES:

Cancun (Quintana Roo)
Ciudad de Mexico (CDMX)

CONEXIONES:

Cancun <---> Ciudad de Mexico

Archivo de columnas

Organiza la información en filas y columnas, lo cual permite abrirla en programas como Excel.

nombre,estado
Cancun,Quintana Roo
Ciudad de Mexico,CDMX

Archivo con etiquetas

Organiza la información mediante etiquetas, una forma común de compartir datos entre programas.

```
<ciudad>  
  <nombre>Cancun</nombre>  
  <estado>Quintana Roo</estado>  
</ciudad>
```

Archivo para aplicaciones web

Organiza la información en un formato ampliamente utilizado en páginas y aplicaciones modernas.

```
{ "nombre": "Cancun", "estado": "Quintana Roo" }
```

6. Aspectos Técnicos de la Implementación

El programa fue construido aplicando el paradigma de Programación Orientada a Objetos (POO), lo cual significa que el código se organiza en clases, cada una encargada de una responsabilidad específica.

Organización de archivos

El proyecto se distribuye en tres archivos, Grafo.h, que contiene las declaraciones de las clases (es decir, qué atributos y qué métodos tiene cada una); Grafo.cpp, que contiene la implementación de dichos métodos; y main.cpp, que contiene la función principal y el menú de interacción con el usuario.

Clase Ciudad

La clase Ciudad hereda de una clase más general llamada Entidad, que únicamente gestiona el identificador numérico. Ciudad agrega los atributos nombre y estado, los cuales se declaran como

```
class Ciudad : public Entidad {  
protected:  
    string nombre;  
    string estado;  
public:  
    Ciudad(int _id, string _nombre, string _estado);  
    string getNombre() const;  
    string getEstado() const;  
    void mostrar() const;  
};
```

protected, es decir, no son accesibles directamente desde fuera de la clase; solo pueden consultarse mediante métodos conocidos como getters. Este diseño es un ejemplo de encapsulamiento, los datos internos de una ciudad están protegidos, y el resto del programa solo puede leerlos a través de funciones específicas, lo cual evita modificaciones accidentales.

Clase Conexion y la clase AristaGrafo

La clase Conexion define los atributos comunes a cualquier enlace entre dos ciudades: el origen, el destino y un estado de actividad. La clase AristaGrafo hereda de Conexion y se especializa en mostrar el enlace utilizando el símbolo de doble flecha, propio de una conexión que funciona en ambos sentidos.

```
void AristaGrafo::mostrarConexion(string o, string d) const {  
    cout << "  " << o << "  " << d  
}
```

Este método se declara como virtual en la clase base Conexion, lo que permite que cada tipo de conexión decida cómo mostrarse en pantalla. A este comportamiento se le conoce como polimorfismo.

Función buscarRuta()

Una de las funciones más relevantes del programa es buscarRuta(), encargada de determinar si existe un camino entre dos ciudades. Primero recorre el arreglo de conexiones buscando un enlace directo; si no lo encuentra, intenta localizar una ciudad intermedia que conecte ambos extremos.

```
void EstructuraGrafo::buscarRuta(int idO, int idD) const {  
    for (int i=0; i<totalA; i++) {  
        int a=aristas[i]->getOrigen(), b=aristas[i]->getDestino();  
        if ((a==idO&&b==idD)|| (a==idD&&b==idO)) {  
            cout<<"Directo: "<<ciudades[iO].getNombre()  
            <<" "<<ciudades[iD].getNombre()<<"\n";  
            return;  
        }  
    }  
}
```

Debemos destacar que la condición $(a==idO \&\& b==idD) || (a==idD \&\& b==idO)$ verifica la conexión en ambos sentidos, reflejando precisamente la naturaleza no dirigida del grafo.

Función eliminarCiudad()

Esta función ilustra cómo se mantiene la integridad de la estructura: antes de eliminar una ciudad, el programa recorre todas las conexiones y elimina aquellas que la involucren, evitando así que queden conexiones que apunten a una ciudad que ya no existe.

```

bool EstructuraGrafo::eliminarCiudad(int id) {
    int idx = indicePorId(id);
    if (idx==-1) return false;
    for (int i=0; i<totalA; i++) {
        if (aristas[i]->getOrigen()==id ||
aristas[i]->getDestino()==id) {
            delete aristas[i];
        }
    }
    return true;
}

```

El uso de delete es necesario porque las conexiones se crean dinámicamente mediante new dentro del método agregarArista(), por lo que liberar su memoria de forma explícita evita fugas de memoria.

Clase EstructuraGrafo y la clase Grafo

EstructuraGrafo es una clase abstracta, lo que significa que no puede crearse un objeto directamente a partir de ella. Define un método virtual puro, mostrarEstructura(), que obliga a cualquier clase derivada a implementarlo según su propio comportamiento. La clase Grafo hereda de EstructuraGrafo y proporciona dicha implementación.

```

virtual void mostrarEstructura() const = 0;

void Grafo::mostrarEstructura() const {
    for (int i=0; i<totalA; i++) {
        aristas[i]->mostrarConexion(
            ciudades[iO].getNombre(), ciudades[iD].getNombre());
    }
}

```

DIGRAFO

1. Introducción

Este reporte describe el funcionamiento de un programa escrito en lenguaje C++ que representa un conjunto de ciudades mexicanas y las conexiones que existen entre ellas, utilizando una estructura de datos conocida como dígrafo, o grafo dirigido. A diferencia de un grafo común, en un dígrafo las conexiones tienen una dirección específica. Esto significa que si una ciudad se conecta hacia otra, no necesariamente existe el camino de regreso entre esas mismas ciudades.

El programa permite al usuario consultar la información de las ciudades y sus conexiones, agregar o eliminar elementos, buscar una ruta entre dos ciudades respetando la dirección de las conexiones, buscar una ciudad por su nombre, y guardar toda la información en distintos tipos de archivo. Internamente, fue construido aplicando Programación Orientada a Objetos, lo cual se detalla más adelante en este documento.

2. ¿Qué es un Dígrafo?

Un dígrafo es una forma de representar un conjunto de elementos junto con las relaciones que existen entre ellos, de manera muy similar a un grafo. La diferencia principal es que, en un dígrafo, cada conexión tiene un sentido definido: parte de una ciudad de origen y llega a una ciudad de destino.

Esto quiere decir que si existe una conexión que va de Cancún hacia la Ciudad de México, esto no implica que exista una conexión que regrese de la Ciudad de México hacia Cancún. Ambos sentidos se consideran conexiones distintas e independientes.

Ejemplo

Cancun ----> Ciudad de Mexico

Esta forma de representar información resulta útil para modelar situaciones donde una relación no es mutua, como calles de un solo sentido, o el caso de seguir a alguien en una red social, donde esa persona no necesariamente sigue de vuelta.

3. Ciudades y conexiones Iniciales

Al iniciar su ejecución, el programa carga automáticamente cinco ciudades mexicanas junto con un conjunto de conexiones dirigidas entre ellas, de manera que el usuario puede comenzar a explorar el programa de inmediato, sin necesidad de capturar información manualmente.

Número	Ciudad	Estado
1	Cancún	Quintana Roo
2	Ciudad de México	CDMX
3	Guadalajara	Jalisco
4	Baja California	Baja California
5	Chiapas	Chiapas

Las conexiones dirigidas precargadas:

- Cancún se dirige hacia Ciudad de México.
- Ciudad de México se dirige hacia Guadalajara.
- Guadalajara se dirige hacia Baja California.
- Baja California se dirige hacia Ciudad de México.
- Ciudad de México se dirige hacia Chiapas.
- Chiapas se dirige hacia Cancún.

Tenemos que resaltar, por ejemplo, aunque Ciudad de México se dirige hacia Chiapas, no existe una conexión registrada en sentido inverso, es decir, de Chiapas hacia Ciudad de México.

4. Funciones del Menú

Ver ciudades

Muestra el listado completo de las ciudades registradas, junto con su número y su estado correspondiente.

Ver conexiones dirigidas

Muestra todas las conexiones existentes en el programa, indicando claramente cuál es la ciudad de origen y cuál la de destino en cada caso.

Buscar ruta

Permite al usuario indicar dos ciudades, una de origen y una de destino, para que el programa determine si existe una conexión entre ellas, respetando siempre la dirección de las conexiones. Primero se revisa si la conexión es directa; si no lo es, el programa intenta encontrar una ruta que pase por una ciudad intermedia, siempre y cuando ambos tramos respeten su dirección correspondiente.

Ejemplo de búsqueda con escala

```
Ruta: Cancun ----> Chiapas
Con escala en Ciudad de Mexico:
Cancun ----> Ciudad de Mexico
Ciudad de Mexico ----> Chiapas
```

Agregar ciudad y/o conexión

Permite registrar una nueva ciudad, una nueva conexión dirigida, o ambas en la misma operación. Al agregar una ciudad, el programa le asigna automáticamente un número de identificación. Al agregar una conexión, se solicitan los números de la ciudad de origen y de la ciudad de destino, en ese orden.

Eliminar ciudad y/o conexión

Permite eliminar una ciudad junto con todas las conexiones que tenga registradas, ya sea como origen o como destino, o bien eliminar únicamente una conexión dirigida en particular, indicando exactamente el sentido de la conexión que se desea eliminar.

Exportar información

Las últimas opciones del menú permiten guardar la información actual del programa en distintos tipos de archivo, de manera individual o todos a la vez. Esta función se describe con más detalle en la sección 6 de este documento.

5. Archivos Generados

El programa permite guardar la información de las ciudades y sus conexiones dirigidas en cuatro tipos de archivo distintos. Cada vez que el usuario agrega o elimina algún elemento, los archivos se actualizan de manera automática.

Archivo de texto

Es un archivo simple, fácil de leer directamente, donde se enlistan las ciudades y sus conexiones dirigidas.

```
CIUDADES:
Cancun (Quintana Roo)
Ciudad de Mexico (CDMX)
```

```
CONEXIONES DIRIGIDAS:  
Cancun ----> Ciudad de Mexico
```

Archivo de columnas

Organiza la información en filas y columnas, lo cual permite abrirla en programas como Excel.

```
nombre,estado  
Cancun,Quintana Roo  
Ciudad de Mexico,CDMX
```

Archivo con etiquetas

Organiza la información mediante etiquetas, una forma común de compartir datos entre programas.

```
<ciudad>  
  <nombre>Cancun</nombre>  
  <estado>Quintana Roo</estado>  
</ciudad>
```

Archivo para aplicaciones web

Organiza la información en un formato ampliamente utilizado en páginas y aplicaciones modernas.

```
{ "nombre": "Cancun", "estado": "Quintana Roo" }
```

6. Aspectos Técnicos de la Implementación

Organización de archivos

El proyecto se distribuye en tres archivos, Digrafo.h, con las declaraciones de las clases; Digrafo.cpp, con la implementación de sus métodos; y main.cpp, con la función principal y el menú de interacción con el usuario.

Clase AristaDigrafo

De forma similar al programa de grafo, la clase Conexion define los atributos comunes a cualquier enlace entre dos ciudades. La clase AristaDigrafo hereda de Conexion, pero a diferencia de AristaGrafo, su método mostrarConexion() utiliza una flecha de un solo sentido, reflejando que el enlace tiene una dirección definida.

```
void AristaDigrafo::mostrarConexion(string o, string d) const {  
    cout << "  " << o << "  ---->  " << d << "\n";  
}
```

Este método se declara como virtual en la clase base Conexion, por lo que cada tipo de conexión decide cómo mostrarse: AristaGrafo usa una doble flecha y AristaDigrafo una flecha simple. A este comportamiento se le conoce como polimorfismo.

Función buscarRuta()

Esta función es la que evidencia con mayor claridad la diferencia frente al programa de grafo. Mientras que en el grafo se aceptaba la conexión en cualquiera de los dos sentidos, en el dígrafo la condición exige que el origen y el destino coincidan exactamente con el sentido registrado.

```
void EstructuraGrafo::buscarRuta(int idO, int idD) const {
    for (int i=0; i<totalA; i++) {
        if (aristas[i]->getOrigen()==idO &&
            aristas[i]->getDestino()==idD) {
            cout<<"Directo: "<<ciudades[idO].getNombre()
            <<" "<<ciudades[idD].getNombre()<<"\n";
            return;
        }
    }
}
```

A diferencia del grafo, aquí la condición solo evalúa `aristas[i]->getOrigen()==idO` seguido de `getDestino()==idD`, sin contemplar el sentido inverso. Esto asegura que el resultado de la búsqueda respete fielmente la dirección de cada conexión registrada.

Función eliminarArista()

Esta función también refleja la naturaleza dirigida de la estructura, mientras que en el grafo se eliminaba la conexión sin importar el orden en que se indicaran las ciudades, en el dígrafo es necesario especificar exactamente el sentido de la conexión que se desea eliminar.

```
bool EstructuraGrafo::eliminarArista(int o, int d) {
    for (int i=0; i<totalA; i++) {
        if (aristas[i]->getOrigen()==o &&
            aristas[i]->getDestino()==d) {
            delete aristas[i];
            return true;
        }
    }
    return false;
}
```

El uso de delete libera la memoria reservada dinámicamente con new al momento de crear la conexión, evitando fugas de memoria dentro del programa.

Clase EstructuraGrafo y la clase Digrafo

De la misma manera que en el programa de grafo, EstructuraGrafo es una clase abstracta que define un método virtual puro llamado mostrarEstructura(). La clase Digrafo hereda de EstructuraGrafo y proporciona su propia implementación, apoyándose en el **polimorfismo de la clase AristaDigrafo para mostrar correctamente cada conexión.**

```
void Digrafo::mostrarEstructura() const {
    for (int i=0; i<totalA; i++) {
        aristas[i]->mostrarConexion(
            ciudades[iO].getNombre(), ciudades[iD].getNombre());
    }
}
```

Árbol Binario de Búsqueda (ABB)

Un Árbol Binario de Búsqueda es un tipo especial de árbol binario que cumple las siguientes reglas:

1. Todos los valores menores al nodo raíz se almacenan en el subárbol izquierdo.
2. Todos los valores mayores o iguales al nodo raíz se almacenan en el subárbol derecho.
3. Cada subárbol también cumple las mismas reglas.

Gracias a esta organización, las búsquedas son más rápidas que en estructuras lineales como arreglos o listas.

El presente programa fue desarrollado en el lenguaje de programación C y tiene como objetivo implementar un Árbol Binario de Búsqueda (ABB). Además de las operaciones básicas de creación e inserción de nodos, el programa permite realizar recorridos en preorden, inorden y postorden, mostrar gráficamente la estructura del árbol en pantalla y almacenar la información contenida en él mediante archivos de texto (.txt) y archivos XML (.xml).

Objetivos Específicos

1. Crear nodos dinámicamente mediante asignación de memoria.
2. Insertar datos respetando las reglas de un árbol binario de búsqueda.
3. Implementar los recorridos Preorden, Inorden y Postorden.
4. Mostrar gráficamente la estructura del árbol.

Un árbol binario es una estructura de datos no lineal compuesta por nodos conectados entre sí mediante enlaces. Cada nodo puede tener como máximo dos hijos:

- Hijo izquierdo.
- Hijo derecho.

Es donde entra la recursividad

Recursividad (una función llamada a sí misma).

```

1  #ifndef ARBOL_H
2  #define ARBOL_H
3
4  typedef struct NodoArbol{
5
6      int dato;
7      struct NodoArbol *izq;
8      struct NodoArbol *der;
9
10 }NodoArbol;
11
12 NodoArbol* crearNodo(int dato);
13
14 NodoArbol* insertar(
15     NodoArbol *raiz,
16     int dato
17 );
18
19 void preorden(NodoArbol *raiz);
20
21 void inorden(NodoArbol *raiz);
22
23 void postorden(NodoArbol *raiz);
24
25 void mostrarArbol(
26     NodoArbol *raiz,
27     int espacio
28 );
29
30 void guardarTXT(NodoArbol *raiz);
31 void escribirTXT(NodoArbol *raiz, FILE *archivo);
32 void guardarXML(NodoArbol *raiz);
33 void escribirXML(
34     NodoArbol *raiz,
35     FILE *archivo
36 );
37
38 #endif

```

El archivo **arbol.h** es el archivo de cabecera del programa. Su función principal es definir la estructura de datos que utilizará el árbol binario y declarar los prototipos de todas las funciones que serán utilizadas en el proyecto.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "arbol.h"
4  #include <stdio.h>
5
6  NodoArbol* crearNodo(int dato){
7
8      NodoArbol *nuevo;
9
10     nuevo = (NodoArbol*)malloc(
11         sizeof(NodoArbol)
12     );
13
14     nuevo->dato = dato;
15     nuevo->izq = NULL;
16     nuevo->der = NULL;
17
18     return nuevo;
19 }
20
21 NodoArbol* insertar(
22     NodoArbol *raiz,
23     int dato
24 ){
25
26     if(raiz == NULL){
27         return crearNodo(dato);
28     }
29
30     if(dato < raiz->dato){
31         raiz->izq = insertar(
32             raiz->izq,
33             dato
34         );
35     }
36     else{
37         raiz->der = insertar(
38             raiz->der,
39             dato
40         );
41     }

```

El archivo **arbol.c** contiene la implementación de todas las funciones declaradas en el archivo de cabecera **arbol.h**. Su principal función es gestionar las operaciones del Árbol Binario de Búsqueda, permitiendo crear nodos, insertar información, recorrer la estructura, visualizarla y almacenar sus datos en archivos TXT y XML.

```

42     };
43 }
44 return raiz;
45 }
46
47 void preorden(NodoArbol *raiz){
48     if(raiz != NULL){
49         printf("%d ", raiz->dato);
50         preorden(raiz->izq);
51         preorden(raiz->der);
52     }
53 }
54
55 void inorden(NodoArbol *raiz){
56     if(raiz != NULL){
57         inorden(raiz->izq);
58         printf("%d ", raiz->dato);
59         inorden(raiz->der);
60     }
61 }
62
63 void postorden(NodoArbol *raiz){
64     if(raiz != NULL){
65         postorden(raiz->izq);
66         postorden(raiz->der);
67         printf("%d ", raiz->dato);
68     }
69 }
70
71 void mostrarArbol(
72     NodoArbol *raiz,

```

Durante el análisis del código se identificaron algunas áreas de mejora que podrían optimizar el funcionamiento y la robustez del árbol binario. Una de las principales fallas encontradas es la falta de validación de memoria al utilizar la función `malloc()`. Actualmente el programa asume que la memoria siempre será asignada correctamente, sin verificar si la reserva fue exitosa. En situaciones donde el sistema tenga poca memoria disponible

Otra limitación importante es que el programa no cuenta con una función para liberar la memoria utilizada por los nodos creados dinámicamente. Cada vez que se inserta un dato se reserva espacio en memoria, pero dicho espacio nunca se libera al finalizar la ejecución. Esto puede generar fugas de memoria, especialmente cuando se trabaja con árboles de gran tamaño o programas que permanecen ejecutándose durante largos periodos.

```

112 if(raiz == NULL)
113     return;
114 fprintf(archivo, "%d\n", raiz->dato);
115 escribirTXT(raiz->izq, archivo);
116 escribirTXT(raiz->der, archivo);
117 }
118
119 void guardarTXT(NodoArbol *raiz){
120     FILE *archivo;
121     archivo = fopen("arbol.txt", "w");
122     if(archivo == NULL){
123         printf("Error al crear archivo\n");
124         return;
125     }
126     fprintf(archivo, "ARBOL BINARIO\n\n");
127     escribirTXT(raiz, archivo);
128     fclose(archivo);
129     printf("\nArchivo arbol.txt generado correctamente\n");
130 }
131
132 void escribirXML(
133     NodoArbol *raiz,
134     FILE *archivo
135 ){
136     if(raiz == NULL)
137         return;
138     fprintf(
139         archivo,
140         "<Numero>%d</Numero>\n",
141         raiz->dato
142     );

```

El programa carece de una función para eliminar nodos. Una vez que un dato es insertado, permanece almacenado hasta que el programa termina su ejecución. La ausencia de esta funcionalidad limita la administración dinámica de la información y reduce la flexibilidad del sistema.

Recorridos del Árbol Binario: Preorden, Inorden y Postorden

Dentro de la implementación del Árbol Binario de Búsqueda se incorporaron tres métodos de recorrido: **Preorden**, **Inorden** y **Postorden**. Estos recorridos permiten visitar todos los nodos del árbol siguiendo diferentes secuencias, dependiendo de la necesidad del programa.

Recorrido Preorden

El recorrido **Preorden** sigue la secuencia:

Raíz → Subárbol Izquierdo → Subárbol Derecho

```
void preorden(NodoArbol *raiz){
```

```
    if(raiz != NULL){
```

```
        printf("%d ", raiz->dato);
```

```
        preorden(raiz->izq);
```

```
        preorden(raiz->der);
```

```
    }
```

```
}
```

Recorrido Inorden

El recorrido **Inorden** sigue la secuencia:

Subárbol Izquierdo → Raíz → Subárbol Derecho

```
void inorden(NodoArbol *raiz){
```

```
    if(raiz != NULL){
```

```
        inorden(raiz->izq);
```

```
        printf("%d ", raiz->dato);
```

```
        inorden(raiz->der);
```

```
    }
```

```
}
```

Recorrido Postorden

El recorrido **Postorden** sigue la secuencia:

Subárbol Izquierdo → Subárbol Derecho → Raíz

```
void postorden(NodoArbol *raiz){
```

```
    if(raiz != NULL){
```

```
        postorden(raiz->izq);
```

```
        postorden(raiz->der);
```

```
        printf("%d ", raiz->dato);
```

```
    }
```

```
}
```

Ejemplo.

```
Practica_Arbol1 - [Practica_Arbol1.dev] - [Executing] - Dev-C++ 5.11
File
  C:\Users\athq\OneDrive\Desktop\Practica_Arbol1.exe
  Cuantos numeros desea ingresar: 7
  Numero 1: 98
  Numero 2: 12
  Numero 3: 45
  Numero 4: 89
  Numero 5: 14
  Numero 6: 56
  Numero 7: 3
  Raiz creada: 98

  PREORDEN:
  98 12 3 45 14 89 56

  INORDEN:
  3 12 14 45 56 89 98

  POSTORDEN:
  3 14 56 89 45 12 98

  ARBOL GRAFICO:
  98
    89
    56
    45
    14
    12
    3

  Archivo arbol.txt generado correctamente
  Archivo arbol.xml generado correctamente

  -----
  Process exited after 18.92 seconds with return value 0
  Presione una tecla para continuar . . .
```

A partir de la salida obtenida puede observarse que el árbol presenta un desbalance considerable. Esto ocurre porque los datos fueron insertados siguiendo un patrón descendente, lo que provoca que cada nuevo nodo se convierta en hijo izquierdo del anterior.

Estos archivos se generan.

Archivo arbol.txt generado correctamente (arbol.txt)

Archivo arbol.xml generado correctamente (arbol.xml)

Conclusiones.

Como equipo, la realización de este proyecto nos permitió comprender de manera práctica el funcionamiento de los grafos dirigidos y la importancia que tienen dentro de la informática para representar conexiones entre diferentes elementos. A lo largo del desarrollo pusimos en práctica conocimientos adquiridos en clase relacionados con estructuras de datos, programación modular, manejo de archivos, validación de información y organización de programas en lenguaje C.

Durante la construcción del sistema aprendimos que un grafo no solamente representa nodos y conexiones, sino que también puede utilizarse para modelar situaciones reales, como rutas de transporte, redes de comunicación o sistemas de distribución. Al agregar nodos, crear rutas con costos y tiempos específicos, generar matrices de adyacencia y almacenar la información en archivos TXT y XML, logramos desarrollar una aplicación funcional capaz de administrar y representar relaciones entre distintos puntos de manera organizada.

Uno de los aspectos más importantes de este proyecto fue el trabajo colaborativo, ya que cada integrante aportó ideas y conocimientos para resolver los problemas que surgieron durante el desarrollo. Esto nos permitió mejorar nuestra capacidad para analizar errores, diseñar soluciones y comprender cómo interactúan los distintos módulos que conforman un sistema completo. Además, reforzamos habilidades relacionadas con la lógica de programación, el uso de funciones, estructuras y la división del código en archivos independientes para facilitar su mantenimiento.

También comprendimos la importancia de validar la información antes de almacenarla, ya que esto ayuda a garantizar la integridad de los datos y evita inconsistencias dentro del sistema. Asimismo, la generación de archivos XML y TXT nos permitió conocer métodos de almacenamiento que pueden utilizarse para intercambiar información entre diferentes aplicaciones o conservar los datos para futuras ejecuciones.

Consideramos que los objetivos planteados al inicio de la práctica fueron alcanzados satisfactoriamente, ya que el programa logró crear el grafo, registrar rutas, mostrar la información almacenada, generar la matriz de adyacencia y exportar los datos correctamente. Sin embargo, reconocemos que existen áreas de mejora, como la implementación completa del algoritmo de Dijkstra, la eliminación de nodos y rutas, la prevención de elementos duplicados y el desarrollo de una interfaz gráfica que facilite la interacción con el usuario.